CSC2001F

Data Structures

Assignment 2

Adam Foflonker

FFLADA001

Foreword

- “make run” builds and runs the app.
- “make instrumented” builds and runs the instrumented version of the app.

Object-Oriented Design

- ***AVLTree*** is a basic generic implementation of an AVL Tree.

- ***AVLTree.Node*** is a node in the ***AVLTree***. A node contains a *key*, *value*, *height* and pointers to the next *left* and *right* nodes.

- The ***Console*** class was written as a wrapper around *System.out* and *System.in*.

- ***GenericsKbAVLApp*** is the main app.

- ***InstrumentationCounter*** class used for the central management of counting comparison operations. This class is used by ***AVLTree***.

- ***KnowledgeBase*** is the abstract base-class for all knowledge base implementations.

- ***KnowledgeBaseAVLTree*** is the AVL Tree implementation of the super-class ***KnowledgeBase,*** which implements the knowledge base as a binary search tree.

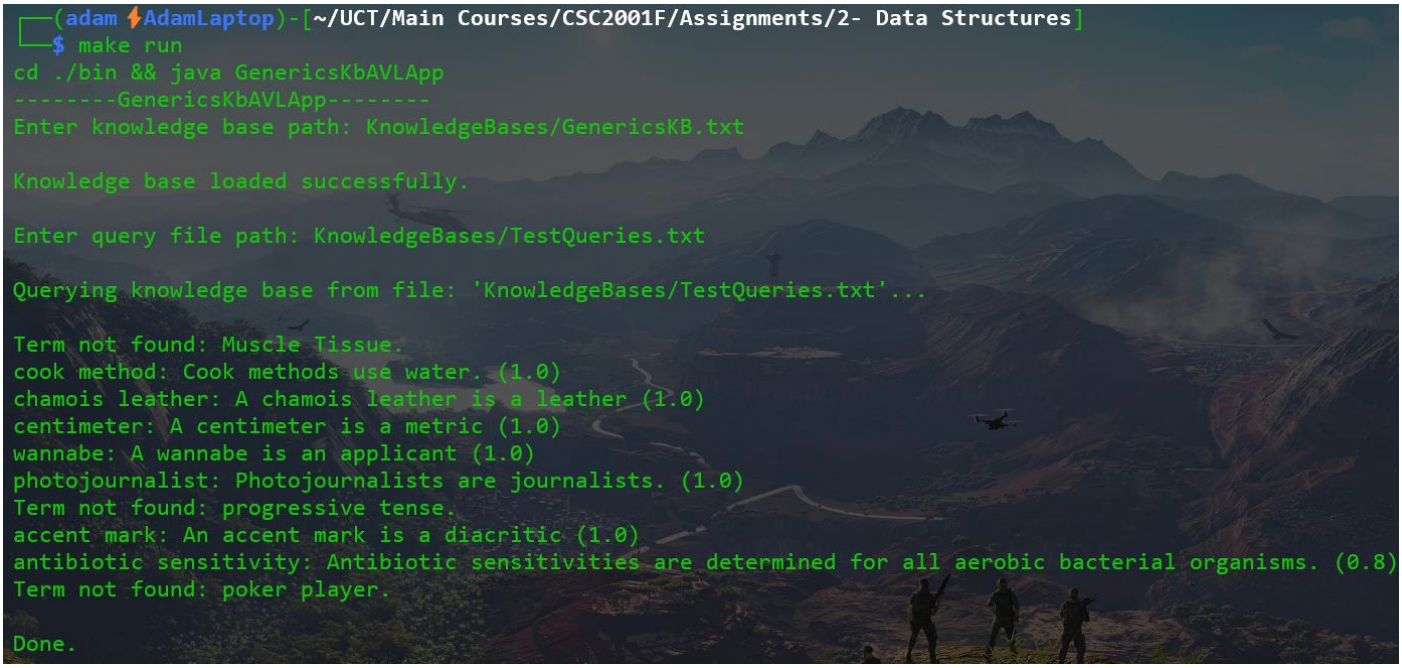
- The ***Statement*** class represents a single statement-entry in the knowledge base which contains a term, sentence and confidence score. A ***KnowledgeBase*** is a collection of ***Statements***.

Program

To ensure that the program was working successfully, a random subset of *GenericsKB-queries.txt,* named *TestQueries.txt*, with 10 queries was used to query *GenericsKB.txt* to test the program.

TestQueries.txt:
Muscle Tissue
cook method
chamois leather
centimeter
wannabe
photojournalist
progressive tense
accent mark
antibiotic sensitivity
poker player

This generates the following output:



Experiment

The goal of this experiment was to compare my **AVLTree** implementation's performance with that of the theoretical performance of algorithms associated with **AVLTrees**.

The experiment was conducted by taking 10 random subsets of *GenericsKB.txt* of varying sizes which were approximately in logarithmic progression with respect to each other. These subsets of the main knowledge base were then read in by the app during 10 different instrumented executions of the app, and then these subsets were queried using *GenericsKB-queries.txt*, which was constant amongst all tests. The amount of comparison operations executed with respect to searching for and inserting keys in the AVL Tree was then recorded at the end of each run. These numbers were obtained by instrumentation using the **InstrumentationCounter** class.

The random subsets of *GenericsKB.txt* were generated by repeatedly executing the following command whilst varying the number 'x' for each different subset of *GenericsKB.txt*

- cat GenericsKB.txt | shuf | head -n x > x.txt

For example, suppose we wanted to generate a random subset of 100 lines from *GenericsKB.txt*:

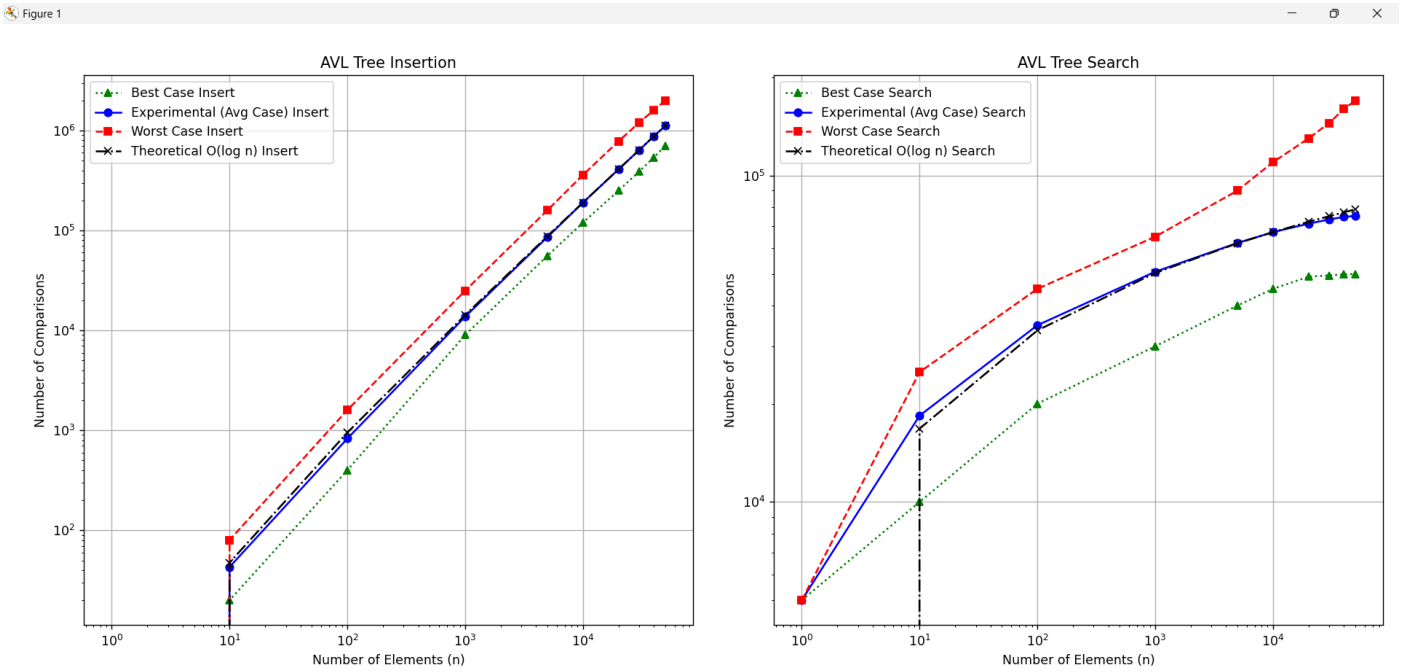
- cat GenericsKB.txt | shuf | head -n 100 > 100.txt

Results

The experiment results, recorded in *Results.txt* are as follows:

n	Insert op-count	Search op-count
1	0	5000
10	43	18386
100	828	34804
1000	13734	50757
5000	86103	62206
10000	190267	67107
20000	408289	71256
30000	637011	73404
40000	878550	74716
50000	1116470	75295

These experimental results can then be used to derive the best and worst-case scenarios for inserting and searching respectively, and can be graphed as seen below:



The shape of these graphs suggests a logarithmic growth as expected, with the experimental and theoretical cases being somewhere in between the best- and worst-case complexities. The experimental case also almost entirely overlaps with the theoretical case which suggests that the experimental values obtained are correct.

Creativity

- Wrote a separate class to centrally handle all instrumentation counting.
- Wrote my own AVL tree implementation which uses generic types and a separately defined explicit key and value. The class is declared as: class AVLTree <_KTy extends Comparable<_KTy>, _Ty>
- Configured GenericsKbAVLApp to accept 1 command-line arg. If no argument is passed, then the app executes normally. If the command-line argument "1" is passed to the app, it will then enable instrumentation and run the instrumented version of the app. This command-line argument is passed to the app by the Makefile when executing "make instrumented"

Git log

0: commit 9a1997fa1c44380c3361b4c60008aa2a19d6ffa9
1: Author: Adam Foflonker <adam.foflonker@gmail.com>
2: Date: Fri Mar 28 20:35:33 2025 +0200
3:
4: Create TestQueries.txt
5:
6: commit ced71166a307d5205d4a609250a12cb39e3604ae
7: Author: Adam Foflonker <adam.foflonker@gmail.com>
8: Date: Fri Mar 28 20:04:32 2025 +0200
9:
…
121: Author: Adam Foflonker <adam.foflonker@gmail.com>
122: Date: Sat Mar 22 18:44:59 2025 +0200
123:
124: Create and implement AVLTree
125:
126: commit 2b075989beee13a90a5357366dec4eb3e3dacbdb
127: Author: Adam Foflonker <adam.foflonker@gmail.com>
128: Date: Sat Mar 22 15:40:41 2025 +0200
129:
130: Add project files